

Refined Publisher Service Specification

Deterministic historical tick replay for Kafka-based simulation

Prepared from the original HLD and the agreed refined replay architecture

Recommended architecture

Two-phase ingest → canonical normalization → deterministic event-time scheduler → partition-affine shard publishers → Kafka → replay lag validation.

Baseline policy: session-aware replay with session rebasing.

Upgrade path: piecewise intraday scaling for open/mid/close realism.

Document

purpose

This specification refines the original publisher-service HLD into a formal replay-system specification that is implementation-ready. It defines the operating contract, inputs, outputs, tunable parameters, chosen design decisions, stage-wise mathematical model, observability requirements, and a coding-agent appendix that can be used to implement the service.

Decision Area	Refined Decision
Replay contract	Session-aware replay with explicit trading calendar and replay horizon
Ingest path	Two-phase ingest: chunked CSV preprocessing to canonical artifact
Normalization	Canonical ordered event stream with stable <code>sequence_id</code> and session metadata
Hot-path scheduler	Deterministic central scheduler with shard publishers
Time mapping	Session rebasing as the production baseline; piecewise scaling as an optional upgrade
Kafka keying	Stable symbol/instrument key mapped to partition-affine publishers
Primary benchmark	Replay lag relative to <code>replay_due_time</code> , not only publish/subscriber elapsed time

Contents

1	Scope, objectives, and non-goals	3
1.1	Primary objectives	3
1.2	Non-goals	3
2	System boundary: inputs, outputs, and tunable parameters	3
2.1	Inputs	3
2.2	Outputs	4
2.3	Tunable parameters that change system behavior	4
3	Recommended design decisions	4
4	Stage 0 – replay contract and calendar policy	5
4.1	Calendar model	5
4.2	Replay timeline model	5
4.3	Session-anchor accumulation	5
4.4	Contract invariants	5
5	Stage 1 – CSV ingestion and storage strategy	6
5.1	Ingestion equations	6
5.2	Artifact selection	6
5.3	Artifact completeness condition	6
6	Stage 2 – sanity check, normalization, and canonical ordering	6
6.1	Required normalized fields	6
6.2	Canonical ordering key	7
6.3	Duplicate and anomaly logic	7
7	Stage 3 – replay scheduling architecture	7
7.1	Scheduler state model	7
7.2	Throughput sufficiency condition	7
7.3	Ordering invariants	7
7.4	Hot-path execution model	8
8	Stage 4 – event-time transformation and event creation	8
8.1	Baseline model: session rebasing	8
8.2	Why session rebasing trims overnight and weekends automatically	8
8.3	Optional upgrade: piecewise intraday scaling	8
8.4	Event envelope	8
9	Stage 5 – Kafka publishing model	9

9.1	Partition mapping	9
9.2	Producer-side constraints	9
9.3	Hot-shard balance	9
10	Stage 6 – metrics, validation, and benchmark methodology	9
10.1	Primary metrics	9
10.2	Order violations	10
10.3	Deterministic-run checks	10
11	Overload policy and operational behavior	10
12	Recommended implementation choices	10
13	Minimal end-to-end mathematical summary	11
14	Appendix A – coding-agent execution prompt	11

1. Scope, objectives, and non-goals

The system replays historical trade ticks from CSV input into Kafka while preserving the statistical and operational properties of market event time as closely as practical. The design optimizes for deterministic replay, per-symbol ordering, session-aware time compression, and measurable replay fidelity under realistic infrastructure limits.

1.1 Primary objectives

- Replay historical tick data from an explicit replay horizon start H rather than from raw historical wall clock time.
- Trim or compress non-trading intervals while preserving intraday session structure for downstream prediction-sensitive consumers.
- Minimize replay error between actual publish time and intended replay due-time.
- Preserve local ordering per instrument/symbol key and maintain reproducible run semantics.
- Separate cold-path preprocessing from hot-path execution so the scheduler does not depend on CSV parsing cost.

1.2 Non-goals

- The service does not attempt to reconstruct exchange internals such as hidden order books, matching-engine queuing, or original network latency.
- The service does not guarantee total global order across all Kafka partitions; the primary invariant is per-key local order.
- The service is not an analytics store. Canonical storage and hot-path replay storage are optimized for replay, not arbitrary ad hoc querying.

2. System boundary: inputs, outputs, and tunable parameters

2.1 Inputs

Input	Description	Required
Historical CSV tick files	Raw trade ticks containing fields such as <code>TRADE_DATE</code> , <code>SYMBOL_CODE</code> , <code>SECURITY_NAME</code> , <code>TICKET_ID</code> , <code>TRADE_PRICE</code> , <code>VOLUME_TRADED</code> , <code>TRADE_VAL</code> , and <code>EXECUTION_TIME</code> .	Yes
Trading calendar	Open/closed dates, session open/close times, half-days, special closures, timezone rules, and optional auction windows.	Yes
Replay configuration	Horizon start H , scaling policy, session gap policy, shard count, Kafka topic, producer settings, and lag thresholds.	Yes
Symbol mapping tables	Optional mapping from security code to canonical symbol or instrument identifier.	Optional
Historical run meta-data	<code>replay_run_id</code> , source dataset version, and normalization manifest for reproducibility.	Recommended

2.2 Outputs

Output	Description
Canonical normalized artifact	A replay-ready dataset with deterministic ordering, canonical timestamps, <code>sequence_id</code> , <code>session_id</code> , intra-session offset, partition key, and quality flags.
Kafka replay event	Published message containing original tick fields plus <code>event_time_original</code> , <code>replay_due_time</code> , <code>publish_time_actual</code> , <code>kafka_ack_time</code> (optional), <code>sequence_id</code> , <code>session_id</code> , <code>scale_policy_id</code> , and <code>replay_run_id</code> .
Run metrics	Replay lag, transport lag, consumer visibility lag, order-violation counts, throughput, queue depth, retry counts, and shard/partition skew metrics.
Validation reports	Preprocessing anomaly reports, calendar mismatches, duplicate counts, malformed-row counts, and deterministic-run manifests.

2.3 Tunable parameters that change system behavior

Parameter	Meaning	Operational effect
H	Replay horizon start timestamp	Shifts the absolute replay clock for all due-times.
<code>Chunk_size</code>	Rows per cold-path CSV chunk	Trades memory footprint against preprocessing throughput.
<code>Artifact_format</code>	Canonical Parquet vs replay shards vs custom binary	Controls cold-path efficiency, hot-path decode cost, and reproducibility.
P	Number of publisher shards / Kafka partitions targeted	Changes load balance, hot-partition risk, and per-shard throughput.
<code>scale_policy</code>	Session rebasing or piece-wise intraday scaling	Changes runtime length, burst preservation, and peak-load profile.
$\alpha_s, \alpha_{s,k}$	Session-wide or segment-wide compression factors	Directly scales replay due-time within sessions or phases.
δ_{gap}	Optional inter-session replay gap	Inserts a small gap between replay sessions if desired.
<code>max_in_flight</code>	Bound on producer requests or outstanding sends	Affects latency, batching, and ordering safety.
<code>lag_thresholds</code>	Warning/fail-fast/catch-up thresholds	Controls overload behavior and quality enforcement.
<code>linger_ms</code> / <code>batch_size</code>	Kafka producer batching controls	Affects batching efficiency and micro-timing fidelity.
<code>tie_break_rule</code>	Ordering for equal timestamps	Determines deterministic canonical sequence on timestamp ties.

3. Recommended design decisions

This specification chooses the following production baseline:

- Stage 0: session-aware replay contract with explicit trading calendar and horizon start H .
- Stage 1: two-phase ingest; parse raw CSV in chunks and write a replay-native normalized artifact.
- Stage 2: canonical normalization with deterministic ordering and stable `sequence_id` assignment.
- Stage 3: deterministic central scheduler with in-process shard publishers; no Celery or external task broker in the hot path.
- Stage 4: session rebasing as the baseline time transform; piecewise scaling remains an optional realism upgrade.
- Stage 5: stable per-symbol Kafka keying and partition-affine shard routing.
- Stage 6: replay lag relative to `replay_due_time` is the primary quality metric.

4. Stage 0 – replay contract and calendar policy

Stage 0 formalizes what faithful replay means before code is written. The scheduler must operate on a replay clock, not on the original historical wall-clock timeline.

4.1 Calendar model

For each session s in the historical range, maintain: session identifier s , trade date d_s , indicator is_open_s , session-open timestamp O_s , session-close timestamp C_s , replay-anchor offset A_s , session scale α_s , and optional phase segmentation metadata. Closed days remain represented in the calendar but contribute zero replay duration.

4.2 Replay timeline model

$$\text{due}_i = H + R(t_i, s(i)) \quad (1)$$

Here t_i is the historical event timestamp, $s(i)$ is the session containing event i , and $R(\cdot)$ is the replay clock transform chosen by Stage 4.

4.3 Session-anchor accumulation

$$A_1 = 0 \quad (2)$$

$$A_s = \sum_{u < s, is_open_u=1} L_u^{replay} + (s - 1) \delta_{gap} \quad (3)$$

L_u^{replay} is the replay duration allocated to open session u . Closed sessions add zero duration. δ_{gap} is optional and may be set to zero for back-to-back session replay.

4.4 Contract invariants

- All events belonging to the same symbol key must preserve local order after normalization and publishing.
- All due-times are derived from the replay calendar, not from sleeping through raw historical time gaps.
- The chosen scale policy must be deterministic for a fixed dataset version and configuration.

5. Stage 1 – CSV ingestion and storage strategy

The cold path reads historical CSV ticks efficiently, validates schema, normalizes field types, and writes a replay-native artifact. The hot path must not depend on text parsing or schema repair.

5.1 Ingestion equations

$$N_{chunks} = \left\lceil \frac{N}{B} \right\rceil \quad (4)$$

$$M_{chunk} \approx B b_{row} + M_{overhead} \quad (5)$$

N is the number of raw rows, B is the chunk size in rows, b_{row} is the average in-memory bytes per normalized row, and M_{chunk} is the approximate working-set memory requirement per chunk.

5.2 Artifact selection

The cold-path artifact may be canonical Parquet for auditability, plus a row-oriented replay shard format for the hot path. A strong practical default is canonical Parquet for normalized storage and length-prefixed Protobuf replay shards for hot execution.

5.3 Artifact completeness condition

$$artifact_row_i = f_{csv}(raw_row_i, calendar, mapping_tables, tie_break_rule) \quad (6)$$

Each raw CSV row is transformed into a canonical normalized row that is already session-aware and partition-aware.

6. Stage 2 – sanity check, normalization, and canonical ordering

Stage 2 produces the canonical event stream that is safe to replay. This stage resolves ordering, canonical field mapping, duplicate handling, sequence assignment, and session membership.

6.1 Required normalized fields

Field	Meaning
<code>event_time_original</code>	Canonical exchange timestamp in the chosen timezone/epoch representation
<code>event_date</code>	Trading date derived from <code>event_time_original</code> and calendar rules
<code>symbol_code</code> / <code>canonical_symbol</code>	Raw and canonical symbol identifiers
<code>trade_id</code>	Original trade identifier if present
<code>sequence_id</code>	Stable replay sequence after normalization
<code>session_id</code>	Calendar session containing the event
<code>relative_offset_in_session</code>	<code>event_time_original - session_open_ts</code>

quality_flags	Duplicate, malformed, late, out-of-session, corrected, or mapping warnings
---------------	--

6.2 Canonical ordering key

$$K_i = (event_time_i, partition_key_i, tie_break_i, raw_row_ordinal_i) \quad (7)$$

$$sequence_id_i = rank_{sorted}(K_i) \quad (8)$$

If two events have the same timestamp, tie_break_i must be deterministic, for example by `trade_id` or raw input ordinal.

6.3 Duplicate and anomaly logic

$$dup(i, j) = \begin{cases} 1 & \text{if } key_{dedup}(i) = key_{dedup}(j) \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

$$quality_flags_i = g(schema_ok_i, in_session_i, duplicate_i, mapping_ok_i, timestamp_ok_i) \quad (10)$$

The implementation should quarantine or flag malformed rows instead of letting them leak into the hot path.

7. Stage 3 – replay scheduling architecture

The key architectural refinement is that the service is scheduled by replay due-time, not by worker availability. One central deterministic scheduler computes or loads due-times, then dispatches due events to partition-affine publisher shards.

7.1 Scheduler state model

$$ready_i(t) = \begin{cases} 1 & \text{if } now(t) \geq due_i \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

$$dispatch_i = \arg \min_i \{ due_i \mid not_yet_dispatched_i \wedge ready_i = 1 \} \quad (12)$$

The scheduler may use a min-heap ordered by due_i , with publisher shards preserving local order per key.

7.2 Throughput sufficiency condition

$$\lambda_{publish}(t) \geq \lambda_{due}(t) \quad \text{for sustained fidelity} \quad (13)$$

Here $\lambda_{due}(t)$ is the required event release rate implied by the replay clock, and $\lambda_{publish}(t)$ is the actual publish capacity. If $\lambda_{publish}$ falls below λ_{due} for long enough, replay lag accumulates.

7.3 Ordering invariants

$$\forall x : sequence_id_i < sequence_id_j \Rightarrow publish_order(i) < publish_order(j) \quad (14)$$

This is the primary order guarantee for events sharing symbol key x . Global order across different partitions is not guaranteed.

7.4 Hot-path execution model

Use in-process threads, goroutines, or async tasks that are directly coupled to the scheduler. Avoid Celery or similar external job systems in the hot path because broker round-trips and background task semantics introduce avoidable timing nondeterminism.

8. Stage 4 – event-time transformation and event creation

Stage 4 maps historical timestamps into replay due-times while preserving intraday structure. This specification chooses session rebasing as the baseline transform and reserves piecewise scaling as an optional upgrade for stronger realism under fixed runtime budgets.

8.1 Baseline model: session rebasing

$$offset_i = t_i - O_{s(i)} \quad (15)$$

$$due_i = H + A_{s(i)} + \alpha_{s(i)} offset_i \quad (16)$$

$O_{s(i)}$ is the historical session-open timestamp for event i , $A_{s(i)}$ is the replay anchor of the containing session, and $\alpha_{s(i)}$ is the session-wide compression factor. If a session is replayed S times faster, then $\alpha_s = 1/S$.

8.2 Why session rebasing trims overnight and weekends automatically

Because A_s is accumulated only from open sessions, dead time never appears in the replay offset. The engine never sleeps through a three-day weekend; it jumps directly from the final due-time of one open session to the replay anchor of the next open session.

8.3 Optional upgrade: piecewise intraday scaling

For event i in segment k of session s :

$$due_i = H + A_s + C_{s,k} + \alpha_{s,k} (offset_i - b_{s,k}) \quad (17)$$

Here $b_{s,k}$ is the historical start offset of segment k , $\alpha_{s,k}$ is the segment-specific compression factor, and $C_{s,k}$ is the cumulative replay duration of all earlier segments in the same session.

$$C_{s,1} = 0 \quad (18)$$

$$C_{s,k} = \sum_{m < k} \alpha_{s,m} (e_{s,m} - b_{s,m}) \quad (19)$$

This keeps the replay timeline continuous while allowing slower compression of the open and close and stronger compression of quieter mid-session intervals.

8.4 Event envelope

Field	Purpose
Original tick payload	Domain data required by consumers
event_time_original	Historical exchange timestamp

<code>replay_due_time</code>	Target publish time on the replay clock
<code>publish_time_actual</code>	Actual send timestamp captured by the publisher
<code>kafka_ack_time</code>	Producer acknowledgement timestamp, if available
<code>sequence_id</code>	Replay ordering identifier
<code>session_id</code>	Replay session identifier
<code>scale_policy_id</code>	Selected scaling policy version
<code>replay_run_id</code>	Run-level provenance for reproducibility

9. Stage 5 – Kafka publishing model

Kafka publishing must preserve the local replay semantics chosen earlier. The system should use stable keys and partition-affine publishers rather than arbitrary fan-out.

9.1 Partition mapping

$$partition_i = h(key_i) \bmod P \quad (20)$$

key_i should normally be the canonical symbol or instrument identifier so that all events for one instrument stay in the same partition for a given run.

9.2 Producer-side constraints

$$in_flight_i \leq max_in_flight \quad (21)$$

$$ack_lag_i = kafka_ack_time_i - publish_time_actual_i \quad (22)$$

Producer settings such as `linger_ms`, `batch_size`, compression, retries, and `max_in_flight` change the trade-off between throughput and micro-timing fidelity. Batching is acceptable, but it must be monitored because extreme batching can destroy the event-time geometry seen by consumers.

9.3 Hot-shard balance

$$skew_ratio = \frac{max_shard_rate}{mean_shard_rate} \quad (23)$$

A large $skew_ratio$ indicates hot partitions or hot symbols and should trigger either rebalancing analysis or higher-capacity shard provisioning.

10. Stage 6 – metrics, validation, and benchmark methodology

The original HLD benchmark based on event-creation time versus publish/subscriber time is useful as a secondary infrastructure metric, but the main quality metric must be replay lag relative to `replay_due_time`.

10.1 Primary metrics

$$replay_lag_i = publish_time_actual_i - replay_due_time_i \quad (24)$$

$$transport_lag_i = kafka_ack_time_i - publish_time_actual_i \quad (25)$$

$$consumer_lag_i = consumer_receive_time_i - replay_due_time_i \quad (26)$$

The service should track replay lag at p50, p95, p99, and max, plus separate lag distributions for open, mid-session, and close periods if piecewise scaling is used.

10.2 Order violations

$$order_violation_count = \sum_x \sum_j \mathbf{1}[sequence_id_{x,j} \leq sequence_id_{x,j-1}] \quad (27)$$

The count should be zero in a healthy run for each symbol-key stream.

10.3 Deterministic-run checks

$$checksum_run = hash(sequence_of(sequence_id, partition_i, replay_due_time_i)) \quad (28)$$

For a fixed dataset version and configuration, repeated dry runs should produce the same canonical checksum before publish.

11. Overload policy and operational behavior

The system must state explicitly how it behaves when replay capacity falls behind due-time demand.

Policy	Behavior	Recommended use
Best-effort continue	Continue publishing while recording degraded quality	Useful for exploratory runs, not for reference-quality replay
Catch-up mode	Reduce sleep precision or drain near-due events faster	Useful when lag excursions are short and recoverable
Fail-fast	Abort run when replay lag exceeds a hard threshold	Recommended for gold-standard validation runs
Micro-batch fall-back	Group a small due-time window and publish together	Useful only when exact micro-timing is no longer achievable

Recommended production default: warn at p95 replay lag above the configured service-level threshold, enter catch-up mode for brief bursts, and fail fast if lag persists beyond the hard limit.

12. Recommended implementation choices

Area	Recommended default	Reason
Cold-path storage	Canonical Parquet plus replay shards	Parquet is excellent for audit/debug; replay shards are better for hot-path sequential execution
Hot-path format	Length-prefixed Protobuf shards	Best engineering balance between speed, maintainability, and schema evolution

Replay policy	Session rebasing first; add piecewise scaling only after stable metrics	Correctness first, realism second
Concurrency model	Central scheduler + in-process shard publishers	Tighter timing control and simpler ordering semantics
Keying strategy	Canonical symbol $\rightarrow$ partition-affine shard	Preserves per-symbol order and aligns with Kafka partitioning
Primary SLA	Replay lag relative to <code>replay_due_time</code>	Measures fidelity to the replay contract rather than only infrastructure latency

13. Minimal end-to-end mathematical summary

For a normalized event i :

$$1) s(i) = \text{session_lookup}(\text{event_time_original}_i) \quad (29)$$

$$2) \text{offset}_i = \text{event_time_original}_i - O_{s(i)} \quad (30)$$

$$3) \text{due}_i = H + A_{s(i)} + \alpha_{s(i)} \text{offset}_i \quad [\text{session rebasing baseline}] \quad (31)$$

$$4) \text{partition}_i = h(\text{key}_i) \bmod P \quad (32)$$

$$5) \text{publish when } \text{now} \geq \text{due}_i \text{ and all prior events for } \text{key}_i \text{ have already been dispatched} \quad (33)$$

$$6) \text{replay_lag}_i = \text{publish_time_actual}_i - \text{due}_i \quad (34)$$

This six-line summary defines the cold-to-hot transformation and the replay fidelity target.

14. Appendix A – coding-agent execution prompt

The following prompt is intended to be passed to a coding agent together with this document. It assumes the agent will implement a replay service from scratch or refactor an existing prototype to match the specification.

You are implementing a deterministic historical tick replay publisher service.

Objective:

Build a service that replays historical trade ticks from CSV into Kafka while preserving session-aware event-time structure.

Required architecture:

1. Cold path:

- Read CSV in chunks.
- Validate schema and normalize types.
- Map raw symbol/security code to canonical symbol when mapping exists.
- Parse timestamps into a single canonical timezone.
- Detect malformed rows and duplicates and attach quality flags.
- Assign stable `sequence_id` using deterministic ordering.
- Join each row to a trading calendar and assign `session_id`.
- Compute `relative_offset_in_session`.
- Write a canonical normalized artifact and a replay shard artifact for hot replay.

2. Trading calendar:

- Represent every date/session explicitly.
- For each open session store `session_id`, `session_open_ts`, `session_close_ts`, `replay_anchor_offset`, `session_scale`.
- Closed days remain represented but contribute zero replay duration.
- Do not infer market state from raw timestamp gaps alone.

3. Hot path:

- Implement a deterministic scheduler that reads normalized ordered events in replay order.
- Use session rebasing as the baseline mapping:
`offset_i = event_time_original_i - session_open_{s(i)}`

- $\text{replay_due_time_i} = H + A_{\{s(i)\}} + \alpha_{\{s(i)\}} * \text{offset_i}$
- Dispatch due events to partition-affine publisher shards.
- Preserve local order per canonical symbol key.
- Avoid Celery or brokered background-job systems in the hot path.

4. Kafka publishing:

- Key by canonical symbol/instrument.
- Route to partition = $\text{hash}(\text{key}) \bmod P$.
- Capture `publish_time_actual` and `kafka_ack_time`.
- Expose retries, queue depth, in-flight requests, and shard skew metrics.

5. Metrics and validation:

- $\text{replay_lag_i} = \text{publish_time_actual_i} - \text{replay_due_time_i}$
- $\text{transport_lag_i} = \text{kafka_ack_time_i} - \text{publish_time_actual_i}$
- Track p50/p95/p99/max replay lag, plus `order_violation_count`.
- Fail validation if per-symbol order is violated.
- Provide a deterministic dry-run checksum over (`sequence_id`, `partition`, `replay_due_time`).

Implementation expectations:

- Produce a clear module structure.
- Separate cold-path preprocessing from hot-path replay.
- Make all tunable parameters configuration-driven.
- Include tests for calendar joins, ordering ties, session trimming, due-time calculation, and per-symbol ordering.
- Include a dry-run mode that computes due-times and metrics without sending to Kafka.
- Prefer clarity and determinism over premature micro-optimization.

Suggested modules:

- config
- calendar
- ingest
- normalize
- shard_writer
- scheduler
- publisher
- metrics
- validation
- cli / main

Deliverables:

- runnable service
- config schema with defaults
- tests
- sample dataset manifest
- README that explains architecture, equations, and operating modes

End of specification.